

```

const PORT = 3000;

// Middleware to parse incoming JSON requests
app.use(express.json());

// Endpoint to handle Docker Hub webhook
app.post('/webhook', (req, res) => {
  console.log('Received webhook:', req.body);

  // Extract repository name from webhook payload
  const repository = req.body.repository.repo_name;

  // Pull the new image from Docker Hub
  exec(`docker pull myusername/${repository}:latest`, (err,
stdout, stderr) => {
    if (err) {
      console.error('Error pulling image: ${stderr}');
      return res.status(500).send('Failed to pull new
image');
    }

    // Restart the Docker container (assuming it's named
'myapp')
    exec('docker stop myapp && docker rm myapp', (err) =>
{
      if (err) {
        console.error('Error stopping container:
${stderr}');
        return res.status(500).send('Failed to stop
old container');
      }

      exec(`docker run -d --name myapp -p 3000:3000 myu
sername/${repository}:latest`, (err) => {
        if (err) {
          console.error('Error starting new
container: ${stderr}');
          return res.status(500).send('Failed to
start new container');
        }

        console.log('Successfully deployed the new
image');
        res.status(200).send('Deployment
successful');
      });
    });
  });
});

// Start the server
app.listen(PORT, () => {

```

```

  console.log(`Server listening on port ${PORT}`);
});

```

By linking your GitHub repository to Docker Hub and configuring automated builds, you enable a smooth continuous deployment process. Every time code changes are pushed to the specified branch, Docker Hub will automatically build a new image. The provided Node.js server listens for webhooks, and can pull the new image and redeploy the application seamlessly.

GitHub Actions for continuous integration and deployment

Creating a CI/CD pipeline with GitHub Actions involves defining workflows in YAML files that specify the steps to be executed in response to certain events. The workflows are stored in the `.github/workflows/` directory of your repository.

- **Define a workflow:** You create a YAML file to define the workflow, including its name, trigger events (like `push` or `pull_request`), and jobs.
- **Specify jobs and steps:** Each job can run in parallel and consists of multiple steps, which can include actions, shell commands, or scripts.

Table 2: Key features of GitHub Actions

Feature	Functionality
Event driven	Triggers workflows based on specific events, such as commits, pull requests, or releases.
Matrix builds	Executes tests or builds across multiple environments or versions of dependencies simultaneously.
Reusable actions	Leverages prebuilt actions from the community to simplify and streamline workflows.
Integration with Docker	Seamlessly builds, tests, and pushes Docker images within GitHub Actions.

Here's an example of a GitHub Actions workflow that automates the process of building a Docker image, running tests, and pushing the image to Docker Hub.

Create a Dockerfile: First, ensure you have a Dockerfile for your application. Here's an example for a simple Node.js application:

```
# Dockerfile
```